LINUX ADMINISTRATION

FINAL EXAM
PRACTICAL LAB PROJECT

QUESTION 1 | TASK 1

Building and Securing a DVWA Testing Environment with Kali Linux

ASSESSMENT ITEM	REQUIREMENT
Assignment type	Individual practical laboratory project
Platforms	Kali Linux and a separate Linux virtual machine hosting DVWA
Official DVWA source	https://github.com/digininja/DVWA
Scope	19 listed security categories; Low, Medium, High, Impossible when supported
Authenticity	Required student ID identity in Linux, terminal, URL, database and application
Recording	One setup video and a separate video for each category; HD 720p minimum
Submission	Accessible Google Drive or YouTube video links and a completed evidence index
Deadline	Monday, 16 November 2026 at 11:59 PM (Rwanda/CAT, UTC+2)
Assessment	100 marks for this task; overall exam weighting set by the lecturer

MANDATORY: EACH STUDENT MUST BUILD, TEST, EXPLAIN AND RECORD THEIR OWN WORK.

Use a private, isolated laboratory only. DVWA must never be exposed to a public network.

Table of Contents

1. Project overview **2**
2. What you must achieve **3**
3. Laboratory safety and academic rules **3**
4. Laboratory architecture and software **3**
4.1. Required architecture 3
4.2. Official DVWA source 4
5. Mandatory student identity and integrity controls **4**
6. Protect the system when customizing credentials **5**
6.1. Manual MariaDB configuration example 5
6.2. Docker Compose configuration example 6
7. Detailed setup and verification procedure **6**
Step 1. Install and identify the VMs 6
Step 2. Keep the previous-lab terminal prompt 6
Step 3. Install DVWA and confirm running services 6
Step 4. Make the exact student-ID domain work 7
Step 5. Set up the database and application login 7
Step 6. Check the advanced challenge prerequisites 7
Step 7. Freeze the verified baseline 7
8. Mandatory vulnerability investigation: 19 categories **7**
9. Security levels and testing method **8**
9.1. Mandatory demonstration sequence for EACH category 9
10. Remediation and verification requirements **9**
11. Mandatory video recording rules **9**
11.1. Number and organization of videos 9
11.2. Quality and authenticity requirements 9
11.3. Suggested order inside each attack video 10
12. Exact required video filenames **10**
13. Submission and access permissions **11**
13.1. Acceptable video hosting 11
13.2. Video privacy 12
14. Assessment rubric for Question 1, Task 1 ..... **12**
15. Final student checklist **12**
Appendix A. Troubleshooting and validation **13**
Appendix B. Useful live verification commands **13**
Appendix C. Official documentation and references **13**

1. Project overview

In this task, you will create a small virtual laboratory, install Kali Linux, deploy the current official Damn Vulnerable Web Application (DVWA), investigate its security weaknesses and show how suitable security controls reduce or prevent those

weaknesses. You must explain both the vulnerable behaviour and the defensive behaviour using your own live demonstrations.

The work is also an assessment of Linux administration skills. Your terminal setup, Linux user account, application address, database and recorded evidence must clearly identify you using the student ID rules in this document. These requirements are mandatory, not optional design choices.

2. What you must achieve

- Install and maintain two Linux-based virtual machines and configure a secure private connection between them.
- Deploy the latest official DVWA source and document the Git commit used for your investigation.
- Keep the Linux user and dynamic terminal prompt from the previous laboratory exercises.
- Personalize the DVWA web address, database, database account and application login using your own student details.
- Investigate all 19 listed DVWA security categories, subject to genuine version-specific availability.
- Compare Low, Medium, High and Impossible where each module supports those levels.
- Explain and demonstrate a practical fix or effective defensive behaviour for each category.
- Produce clear, narrated, student-authenticated HD videos and submit working viewing links.

3. Laboratory safety and academic rules

1. Only test the DVWA environment that you created or that the lecturer expressly authorized.
2. Use host-only or internal VM networking for security testing. Do not use bridged networking, public IP addresses or external port forwarding.
3. A separate temporary NAT adapter may be used for downloading software. Disconnect unnecessary external connectivity before testing.
4. Do not attempt attacks against other students, campus networks, workplaces, websites or public services.
5. Use laboratory-only accounts and harmless test data. Do not use real personal information or real credentials.
6. Do not perform denial-of-service, persistence, data theft, destructive actions or attacks outside the challenge modules.
7. Keep VM snapshots and backups before making database or application changes.
8. Every command, finding and explanation submitted must be your own. You may consult documentation, but you must understand what you demonstrate.

The required exam password format is predictable. Use it ONLY in this disposable classroom laboratory; never reuse it on personal, campus or production systems.

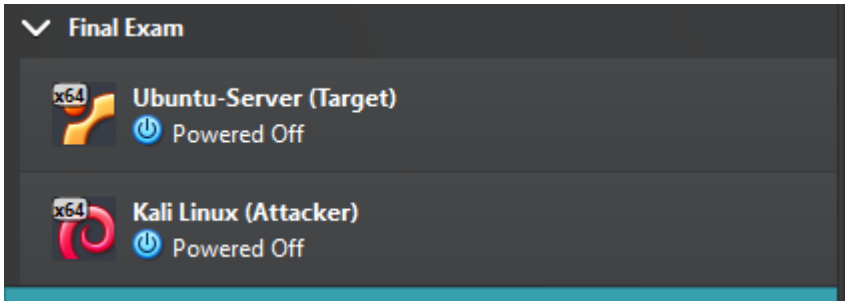
4. Laboratory architecture and software

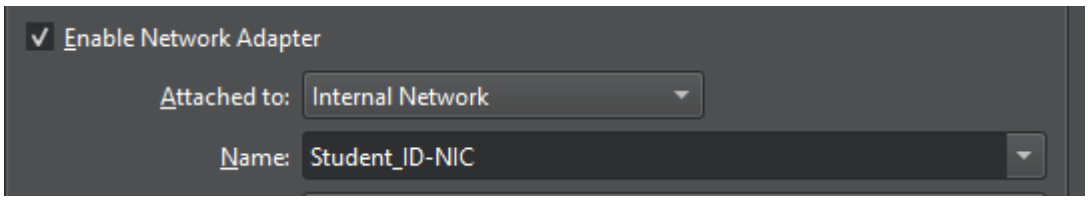
4.1. Required architecture

MACHINE	ROLE	MUST SHOW
Kali Linux VM	Student testing workstation	Student-ID Linux user, previous-lab terminal prompt, browser, command tools, private IP address
Linux DVWA VM	Target application server	Student-ID Linux user, private IP address, current DVWA, web service and MariaDB
Private virtual network	Connects only the permitted laboratory systems	Successful Kali-to-DVWA communication without any internet-facing DVWA service

Recommended hypervisors include VirtualBox and VMware. Ubuntu Server or another suitable Linux distribution can host DVWA. Docker Compose or a documented Apache/PHP/MariaDB installation can be used. You need a screen recorder that supports simultaneous webcam and microphone capture.

Example private addresses (Virtual Internal Network Interface: Student_ID NIC) Kali Linux (**Attacker**) 192.168.1.10/24 and DVWA server (**Victim/Target**) 192.168.1.20/24.





4.2. Official DVWA source

```
git clone https://github.com/digininja/DVWA.git
cd DVWA && git rev-parse --short HEAD && git log -1 --format=%cs
```

Use the up-to-date source from the official repository. Record the commit hash and commit date in your setup video and evidence index. Do not use an outdated course video, mirror or unofficial image as your authoritative source.

5. Mandatory student identity and integrity controls

Your real Student_ID is the identity reference for this task. Replace the placeholders below with your own values consistently. Display your identity in the terminal, browser and evidence index. Students who share a computer must still use separate Linux accounts, URLs, databases, recordings and submissions.

ITEM	REQUIRED STUDENT-SPECIFIC VALUE
Linux login user, on BOTH VMs	Student_ID, as established in previous laboratories
Existing terminal format	Student_ID and CURRENT timestamp, updated dynamically
VM names / hostnames	kali-Student_ID and dvwa-Student_ID (safe hostname form)
DVWA browser domain	task1.student_ID.linux-f26
Full preferred DVWA address	http://task1.student_ID.linux-f26/
MariaDB database name	db_Student_ID
MariaDB application DB username	first letter of first name + entire last name, lowercase
MariaDB application DB password	Student_ID_Linux-f26
DVWA web application login username	Student_ID
DVWA web application login password	Student_ID@linux-f26

ITEM	REQUIRED STUDENT-SPECIFIC VALUE
Video filenames and evidence index	Student_ID at the start of every name and each file entry

Example: For the sample name Joshua IRADUKUNDA, the required database username is jiradukunda. If a student ID were Student_ID, the database would be db_Student_ID, the database password would be Student_ID@linux-f26 and the local DVWA address would be http://task1.Student_ID.linux-f26/. This is only an example. Do not copy the sample identity.

Important: The DB username and the DVWA web login username are different accounts. The DB user connects the PHP application to MariaDB; the web login identifies a user inside DVWA.

Some student IDs contain characters that Linux usernames, DNS labels or SQL identifiers do not accept. Use the exact ID where valid. If it is not valid, obtain lecturer approval for ONE documented normalization before making changes. Never change or shorten the ID silently.

6. Protect the system when customizing credentials

These changes can prevent DVWA from loading or connecting to its database if even one value is wrong. They are still mandatory. Complete the personalization carefully, take a snapshot and confirm that the application works again before vulnerability testing.

1. Plan all names and credentials first. Write down the required values in your private lab notes and check spelling, upper/lowercase and special characters.
2. Take a VM snapshot and back up any work or sample database that you need to preserve.
3. Create the database and DB user, assign only the permissions required for that dedicated lab database, and test the account.
4. Update the active DVWA configuration or equivalent environment variables so that the DB name, DB username and DB password match the database server.
5. Initialize or reset the DVWA database only when necessary. DVWA setup can erase the selected lab database; do not aim it at any database containing other data.
6. Create or safely change a DVWA web login to Student_ID with the assigned application password; do not confuse the login account with the MariaDB service account.
7. Confirm a successful live web login, normal application operation and the correct database connection.
8. Recheck the configuration after a container rebuild, image update, database reset or VM restoration. Take a second clean snapshot when all changes work.

6.1. Manual MariaDB configuration example

This illustration uses Student_ID Student_ID and the sample database username jiradukunda. Replace all values. Use the appropriate database administrator account for your own Linux installation.

```
sudo mariadb
CREATE DATABASE db_Student_ID;
CREATE USER 'jiradukunda'@'localhost' IDENTIFIED BY 'Student_ID_Linux-f26';
GRANT ALL PRIVILEGES ON db_Student_ID.* TO 'jiradukunda'@'localhost';
SHOW DATABASES;
```

For a manual installation, set the active config/config.inc.php values db_database, db_user, db_password and db_server to match your database setup. Keep the service account local to the database host where possible. Use a secure method to supply passwords rather than placing them in command history outside this lab.

6.2. Docker Compose configuration example

For Docker, inspect compose.yml and the live configuration carefully. The database service uses initialization variables MYSQL_DATABASE, MYSQL_USER and MYSQL_PASSWORD. The DVWA application separately reads DB_DATABASE, DB_USER and DB_PASSWORD (or equivalent PHP config values). Both sides must match. DB_SERVER should still resolve to the database container service, commonly db.

MYSQL_DATABASE=db_Student_ID

MYSQL_USER=jiradukunda

MYSQL_PASSWORD=Student_ID@linux-f26

DB_DATABASE=db_Student_ID | DB_USER=jiradukunda

DB_PASSWORD=Student_ID@linux-f26 | DB_SERVER=db

WARNING: MariaDB initialization variables normally apply when a new empty data volume is created. Editing them after the database already exists does not automatically rename the database or update existing users. Modify the existing database safely or restore a clean snapshot before reinitialization. Never delete a volume with work you still need.

Application login changes must also be preserved in the working DVWA user database. The stock DVWA setup process can restore its example accounts. If a challenge needs an original sample account, retain it for that challenge while ensuring your personalized Student_ID login works as well.

7. Detailed setup and verification procedure

Step 1. Install and identify the VMs

1. Install Kali Linux and a separate Linux server virtual machine.
2. Name the VMs and hostnames using the Student_ID convention.
3. Log in with your existing Student_ID Linux user on both machines. Show whoami, id and hostnamectl.
4. Confirm the private IP addresses, routes and connectivity. Explain the virtual network type.

whoami | id -un | hostnamectl | ip -br addr | date "+%F %T %Z"

Step 2. Keep the previous-lab terminal prompt

The lecturer already required a personalized terminal in earlier labs. Do not replace it with the default root, kali or ubuntu prompt. Continue using the SAME approved prompt format, showing your Student_ID and the current date and time. The timestamp must be refreshed when the prompt is displayed. It must not be a fixed text label.

whoami # must display your approved Student_ID user

date "+%Y-%m-%d %H:%M:%S %Z" # compare with terminal timestamp

Show the prompt in the setup video and again at the beginning of every vulnerability video. If running a command with sudo temporarily changes the context to root, return to your Student_ID shell before continuing the narrated demonstration.

Step 3. Install DVWA and confirm running services

git clone https://github.com/digininja/DVWA.git

cd DVWA && git rev-parse --short HEAD

docker version | docker compose version

docker compose up -d | docker compose ps

The official Compose example publishes DVWA by default on localhost port 4280. A Kali VM on a separate guest cannot reach that localhost binding. Configure the web service ONLY on the DVWA VM private interface, or use a reverse proxy restricted to that interface. Do not bind DVWA on 0.0.0.0 or on any internet-facing host interface.

Example lab-only Compose binding: 192.168.1.20:4280:80

Step 4. Make the exact student-ID domain work

A DNS name is not an IP address. Keep the DVWA server on its private IP address and configure a local name-resolution entry or an internal DNS record to point the required name to that IP. For a small host-only lab, /etc/hosts is acceptable.

Kali /etc/hosts example: 192.168.1.20 task1.Student_ID.linux-f26

The REQUIRED main URL should load without a port suffix: http://task1.Student_ID.linux-f26/. You may use a lab-only reverse proxy on private port 80 to forward requests to the DVWA container on port 4280, or a correctly restricted Apache setup. A URL with :4280 is acceptable for troubleshooting, but the final recorded main address must use the required hostname without a visible nonstandard port.

getent hosts task1.Student_ID.linux-f26

curl -I http://task1.Student_ID.linux-f26/

The same Student_ID domain must appear in the browser during every vulnerability video. Keep this name inside the isolated VM environment. It is not a public DNS registration.

Step 5. Set up the database and application login

Apply Section 6 before running the challenge modules. Verify the database name and username, show the successful database connection without exposing unrelated secrets, and demonstrate that you can log into DVWA with the Student_ID web account. Do not leave authentication disabled for the final baseline.

Step 6. Check the advanced challenge prerequisites

The API Playground may require Apache mod_rewrite and PHP Composer dependencies. Follow the current repository README for the build installed. In a manual Apache installation, confirm rewrite support and the installed Composer packages in vulnerabilities/api. In Docker, inspect the image and troubleshoot any missing features without making the lab public.

The Insecure CAPTCHA challenge may need test keys; explain any controlled, documented setup limitation. Record the actual installed module menu and do not invent successful tests for features that genuinely cannot run.

Step 7. Freeze the verified baseline

1. Open the main DVWA page from Kali by the exact Student_ID domain.
2. Confirm the personalized DVWA web login works.
3. Show the correct database connection, the private listening interface and a successful VM connection test.
4. Show the installed commit reference and the list of visible challenge modules.
5. Create a snapshot called STUDENT_ID_Q1_T1_BASELINE or an equivalent clearly identified snapshot.

8. Mandatory vulnerability investigation: 19 categories

Test every category below using the newest official DVWA build. Check the actual module names and available security levels in your installed copy. The list is the required exam scope. Where a level or feature is genuinely unsupported, explain the technical reason and record N/A rather than inventing evidence.

NO.	DVWA CATEGORY	RESEARCH AND DEFENSE FOCUS
01	Brute Force	Authentication strength, login throttling and account protection. Rate limiting, lockout and MFA concepts.

NO.	DVWA CATEGORY	RESEARCH AND DEFENSE FOCUS
02	Command Injection	Unsafe handling of input passed to system commands. Avoid shell execution; strict input handling.
03	CSRF	Requests issued without valid user intent. Per-request anti-CSRF tokens and SameSite controls.
04	File Inclusion	Untrusted paths used to include application content. Allowlist permitted files, avoid dynamic inclusion.
05	File Upload	Improper validation of files sent to the server. Type validation, storage isolation and execution prevention.
06	Insecure CAPTCHA	Weak server-side enforcement of verification steps. Validate challenges on the server.
07	SQL Injection	Unsafe concatenation of input into database queries. Prepared statements and limited DB permissions.
08	SQL Injection (Blind)	Data inference through indirect query behaviour. Prepared statements and safe error handling.
09	Weak Session IDs	Predictable or weak session identifiers. CSPRNG identifiers and secure cookie settings.
10	XSS (DOM)	Unsafe client-side DOM insertion. Safe DOM APIs and contextual encoding.
11	XSS (Reflected)	User input returned unsafely in HTML. Output encoding and safe templating.
12	XSS (Stored)	Stored user content rendered without encoding. Output encoding, content rules and CSP.
13	CSP Bypass	Failures or misconfigurations in CSP protections. Restrictive CSP and correct nonce use.
14	JavaScript	Weak client-side checks or trust assumptions. Enforce validation and authorization server-side.
15	Authorisation Bypass	Unauthorized access to protected operations. Server-side role and permission checks.
16	Broken Access Control	Access to resources outside assigned privileges. Object-level access checks and deny by default.
17	Open HTTP Redirect	Untrusted redirect destinations. Allowlist destinations; validate relative paths.
18	Cryptography	Weak or unsafe cryptographic practice. Modern cryptographic libraries and sound key handling.
19	API Security (API Playground)	Weak API authentication, object access or input rules. Authorize each operation, validate data, rate-limit.

API Playground may consist of several sub-tests. Include those available in your installed build in the API video. DVWA versions may differ in which modules support each global security level.

9. Security levels and testing method

LEVEL	WHAT TO DO	REQUIRED PROOF
Low	Explain the weakness; perform a harmless controlled test.	Working vulnerable behaviour and relevant request or code.
Medium	Repeat the same objective and identify changed checks.	Live comparison and your explanation.

LEVEL	WHAT TO DO	REQUIRED PROOF
High	Test the more restrictive implementation and identify any remaining weakness.	Live outcome, not a guessed result.
Impossible	Study the secure reference and retest the same objective, if supported.	Defensive code/control and live blocked outcome or explained N/A.

9.1. Mandatory demonstration sequence for EACH category

1. Start the separate video with your full name and Student_ID; show the current Student_ID terminal and DVWA hostname.
2. Explain the vulnerability, a realistic impact and the normal purpose of the application feature.
3. Select Low and show a controlled, harmless test with its live outcome.
4. Select Medium and High in turn, showing what changes and what does not change.
5. Select Impossible, when available, and demonstrate the secure control with a comparable test.
6. Show the relevant source, request, response, log or terminal output and explain the root cause.
7. Describe the exact fix, such as parameterized queries, output encoding or server-side authorization.
8. If Impossible is not offered, demonstrate the tested fix in your own isolated patched copy or explain any approved technical limitation.
9. Finish by comparing the before/after results and stating what you learned.

A failed test may be valid evidence of an effective control. Explain the actual result. Do not claim every module has identical security-level behaviour or that Impossible always eliminates every possible risk.

10. Remediation and verification requirements

An attack demonstration alone is not sufficient. For all 19 categories, explain the defensive control and verify it when practical. Distinguish between merely changing the DVWA security level and modifying vulnerable source code or configuration. Demonstrate what you actually changed and do not describe DVWA Impossible as a production security certificate.

- Record the cause, affected input or function, observable behaviour and likely impact.
- Identify the secure control and why it addresses the root cause.
- Show the before-and-after source or configuration when appropriate.
- Run a comparable safe retest and explain the observed difference.
- Confirm the application still works for legitimate requests and that the database is not damaged.
- Use snapshots to restore the original challenge after destructive or risky changes to your own lab.

11. Mandatory video recording rules

11.1. Number and organization of videos

You must record one complete environment setup video, followed by one separate video for EACH of the 19 security categories in Section 8. This gives 20 videos when all 19 categories are available. Do not combine different attacks into a single recording or submit one long video for the whole project.

11.2. Quality and authenticity requirements

1. Video resolution must be at least HD 720p (1280 x 720). Higher resolution is encouraged when text remains readable.

2. Your FACE must be continuously visible in webcam picture-in-picture from the beginning to the end of EVERY video. Do not replace it with a profile picture or a frozen frame.
3. Clearly say your full name and Student_ID at the beginning of each video and also display your identity on screen.
4. Show the Student_ID Linux shell prompt and live current timestamp at the beginning of each video. Keep the student-ID DVWA URL visible when demonstrating the browser.
5. Record actual terminal commands and outputs at every relevant stage. Show the process, not only screenshots or final results.
6. Show the result loading or changing LIVE in the browser or terminal. Still images alone do not count as proof.
7. Use your own understandable voice narration throughout. Explain what you are doing, why you are doing it and what the result means.
8. For each attack video, show the security levels, the observed differences, the explanation of the fix and a defensive retest.
9. Use MP4 format with stable video, clear focus, readable fonts and audio free of serious distortion.
10. Play back the full video before submitting. Videos with unreadable screen content, an inaudible or missing voice, or no continuous face view will receive ZERO for that affected video.

Editing may be used to remove long installation downloads or silence, but it must not hide required commands, failure messages, vulnerability level changes, or live demonstrations. Do not splice another student's work into your recording.

11.3. Suggested order inside each attack video

PART	CONTENT TO SHOW
A. Identity	Student name, Student_ID, face, dynamic terminal timestamp and required website hostname.
B. Introduction	Problem explained in clear English; why the normal feature matters.
C. Live tests	Low, Medium and High where available, with actual requests and visible responses.
D. Secure comparison	Impossible or an effective verified fix, with source/control and live retest.
E. Summary	One short conclusion explaining cause, mitigation and what was learned.

12. Exact required video filenames

Replace Student_ID with your assigned student registration number. Keep the numbering, spellings and topic order below. All filenames must begin with the same Student_ID.

VIDEO	FILENAME
00	Student_ID_Q1_T1_00_Lab_Setup.mp4
01	Student_ID_Q1_T1_01_Brute_Force.mp4
02	Student_ID_Q1_T1_02_Command_Injection.mp4
03	Student_ID_Q1_T1_03_CSRF.mp4
04	Student_ID_Q1_T1_04_File_Inclusion.mp4
05	Student_ID_Q1_T1_05_File_Upload.mp4

VIDEO	FILENAME
06	Student_ID_Q1_T1_06_Insecure_CAPTCHA.mp4
07	Student_ID_Q1_T1_07_SQL_Injection.mp4
08	Student_ID_Q1_T1_08_SQL_Injection_Blind.mp4
09	Student_ID_Q1_T1_09_Weak_Session_IDs.mp4
10	Student_ID_Q1_T1_10_XSS_DOM.mp4
11	Student_ID_Q1_T1_11_XSS_Reflected.mp4
12	Student_ID_Q1_T1_12_XSS_Stored.mp4
13	Student_ID_Q1_T1_13_CSP_Bypass.mp4
14	Student_ID_Q1_T1_14_JavaScript.mp4
15	Student_ID_Q1_T1_15_Authorisation_Bypass.mp4
16	Student_ID_Q1_T1_16_Broken_Access_Control.mp4
17	Student_ID_Q1_T1_17_Open_HTTP_Redirect.mp4
18	Student_ID_Q1_T1_18_Cryptography.mp4
19	Student_ID_Q1_T1_19_API_Security.mp4

13. Submission and access permissions

The final deadline is MONDAY, 16 NOVEMBER 2026 at 11:59 PM (CAT, UTC+2). The lecturer will provide the official submission form or platform. Submit an evidence index containing all 20 numbered entries and a working link for each video.

13.1. Acceptable video hosting

1. Google Drive: upload each MP4, set General access to Anyone with the link and permission to Viewer, and copy the viewer link.
2. YouTube: use Unlisted (recommended) or Public. Both allow a person with the URL to view the recording without a specific invitation.
3. YouTube Private is NOT generally accessible to anyone with a link. If you choose Private, you must explicitly invite the lecturer's authorized account and confirm the lecturer can open it. Otherwise use Unlisted.
4. Check each link in a private/incognito browser window or with a permitted viewer account. A link that asks the lecturer to request access is not accepted.
5. Keep the files available for assessment and any academic integrity verification required under institution policy.

A short evidence index must show your full name, Student_ID, DVWA Git commit, private lab domain, the 20 filenames, each viewing URL, a Low/Medium/High/Impossible coverage status and a short explanation of any N/A item. The index is a supporting document, not a substitute for the videos.

13.2. Video privacy

Because your recordings contain your face, student details and a deliberately insecure lab, consider link-based viewing rather than public search listing. Do not record personal passwords, banking data, other students, or information unrelated to your lab. The lab password specified in this task must never be reused anywhere else.

14. Assessment rubric for Question 1, Task 1

ASSESSMENT COMPONENT	MARKS
Private network, VM setup, services and current DVWA installation	12
Student_ID Linux user, terminal and live timestamp	10
Student_ID URL, DNS, database and web login personalization	14
Nineteen-category coverage and level-by-level analysis	24
Technical explanation, remediation and verified retesting	20
Individual HD narrated PiP recordings with live evidence	12
Correct video naming, access links, index and organization	8
TOTAL	100

The lecturer may assign zero for an affected video if its screen or voice cannot be understood, if the student's face is missing, or if the recording is not authentic. Work outside the permitted lab or fabricated evidence is subject to academic integrity rules. The final course examination weight for this task follows the lecturer's exam plan.

15. Final student checklist

- ☐ I installed Kali Linux and a separate DVWA Linux server on an isolated virtual network.
- ☐ I used the current official DVWA code and recorded the Git commit.
- ☐ Both Linux sessions use my Student_ID user and the previous-lab terminal with a live timestamp.
- ☐ My main browser URL works as http://task1.Student_ID.linux-f26/ from Kali.
- ☐ My DB name is db_Student_ID; my DB username follows the name rule; my DB password is correct.
- ☐ My Student_ID web login works, and all application/database changes are documented.
- ☐ I verified that DVWA remains functional and not publicly accessible.
- ☐ I tested or justified N/A for all 19 modules and relevant security levels.
- ☐ Every module video explains the weakness, shows live evidence and verifies a suitable control.
- ☐ I created 20 correctly named MP4 videos with continuous face PiP, voice narration and 720p minimum resolution.
- ☐ I tested every Google Drive or YouTube link without needing permission requests.
- ☐ I submitted the complete evidence index and all working links by 16 November 2026, 11:59 PM CAT.

Appendix A. Troubleshooting and validation

ISSUE	WHAT TO CHECK
Site opens by IP but not by domain	Check Kali /etc/hosts or private DNS, lowercase Student_ID label, getent hosts, and the exact browser URL.
Domain resolves but port 80 fails	Check private port 80 web binding or lab reverse proxy, private firewall and the backend port 4280.
Kali cannot reach DVWA	Verify both VMs use the same internal/host-only network and that Docker listens only on the target private IP.
Database access denied	Check db name, DB_USER, DB_PASSWORD, DB_SERVER, MariaDB user host, grants and matching case.
Database name unchanged after Compose edit	The existing MariaDB volume may retain original initialization. Use a safe migration or restore a clean lab snapshot.
DVWA web login stopped working	Check the DVWA user record, password hash handling, whether setup/reset restored defaults, and that authentication is enabled.
API Playground fails	Check current module dependencies, Apache rewrite support and PHP Composer package installation.
Terminal timestamp does not change	Check the actual previous-lab shell prompt, not a static echo or manually typed date.
Link asks for permission	Use Drive Anyone with the link / Viewer or YouTube Unlisted/Public; verify as an outside viewer.
Video readable but silent	Check microphone capture and play the full MP4 before publishing.

Appendix B. Useful live verification commands

COMMAND	WHAT IT PROVES
whoami | id -un	Current Linux login identity.
date "+%F %T %Z"	Actual Linux system timestamp.
hostnamectl	Guest hostname and Linux system information.
ip -br addr | ip route	Private IP configuration and route.
getent hosts task1.STUDENT_ID.linux-f26	Local DNS or /etc/hosts resolution (replace placeholder).
curl -I http://task1.STUDENT_ID.linux-f26/	HTTP service responds by required name (replace placeholder).
git rev-parse --short HEAD	Installed DVWA source commit hash.
docker compose ps | docker compose logs --tail=30	Current DVWA container/service status.
ss -ltn	Listening network interfaces and ports.
sudo mariadb -e "SHOW DATABASES;"	Database inventory; only in an authorized local lab.

Appendix C. Official documentation and references

- DVWA official source, README and installation instructions: https://github.com/digininja/DVWA
- DVWA official Docker Compose file: https://github.com/digininja/DVWA/blob/master/compose.yml
- DVWA active configuration template: https://github.com/digininja/DVWA/blob/master/config/config.inc.php.dist
- Kali Linux virtualization documentation: https://www.kali.org/docs/virtualization/
- Docker Engine installation guide: https://docs.docker.com/engine/install/

- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- **YouTube Tutorials:**
  - https://www.youtube.com/watch?v=sncUAUVN2_s&list=PLUu16IFJeuvuKfElUcsMjQwqZgJHKD1BV&index=51
  - https://www.youtube.com/watch?v=GmWQ1VIjd2U&list=PLHUKi1UlEgOJLPSFZaFKMoexpM6qhOb4Q

END OF QUESTION 1, TASK 1